

Visual Exploratory Data Analysis (VisEDA): PART E

Text and NLP Exploratory Data Analysis

A Guide to Text and Natural Language Dataset Analysis

Isaac Osei Agyemang

Version 1.0.0

2026

Preface

Exploratory Data Analysis is as important for natural-language data as it is for images, videos, hyperspectral cubes, and point clouds. Text datasets can contain class imbalance, empty records, inconsistent encodings, unusually short or long documents, duplicate content, repeated boilerplate, mixed writing scripts, URLs, email addresses, hashtags, mentions, numerical artefacts, and large differences in vocabulary or readability. These properties can change model behaviour before any tokeniser, embedding model, recurrent network, or transformer is trained.

TextEDA extends VisEDA to text and NLP datasets using the same progressive workflow used elsewhere in the library: load the dataset, obtain a structured numerical summary, inspect visual diagnostics, and export a shareable HTML report. It supports plain-text documents, web text, delimited datasets, JSON/JSONL corpora, and in-memory strings. The default implementation deliberately uses lightweight regular-expression tokenisation rather than silently requiring NLTK, spaCy, or transformer packages.

This manual documents the current TextEDA implementation. It explains both the public API and the calculations behind the reported metrics so that results can be interpreted correctly and reproduced in downstream NLP workflows.

Isaac Osei Agyemang

August 2026

Table of Contents

Preface	2
Table of Contents	3
Chapter 1 — Introduction	6
1.1 What is VisEDA TextEDA?	6
1.2 Why Text EDA Matters	6
1.3 Design Philosophy	6
1.4 Installation	6
1.5 Supported Data Formats	7
1.6 Tokenisation and Language Scope	7
Chapter 2 — Quick Start	8
2.1 Directory Dataset with Parent-Folder Labels	8
2.2 CSV, TSV, JSON, and JSONL Datasets	8
2.3 In-Memory Text	8
2.4 Getting a Summary	8
2.5 Vocabulary and N-gram Analysis	9
2.6 Document Similarity and Near-Duplicates	9
2.7 Generate Visualisations	9
2.8 Generate an HTML Report	9
2.9 Command-Line Interface	9
2.10 User-Dataset-Only Test Script	10
Chapter 3 — The TextEDA Class	11
3.1 Constructor	11
3.2 load() — Files, Lists, and Directories	11
3.3 load_texts() — In-Memory Strings	11
3.4 get_record() and get_text()	11
3.5 vocabulary() — Corpus Token Frequencies	12
3.6 summary() — Full Dataset Statistics	12
3.7 pairwise_document_distances()	12
3.8 near_duplicate_pairs()	12
3.9 report() — Self-Contained HTML	12
Chapter 4 — Input Parsing and Dataset Organisation	13
4.1 Path Resolution	13
4.2 Encoding Detection	13
4.3 Plain Text and Markup Files	13
4.4 CSV and TSV Field Selection	13
4.5 JSON and JSONL Expansion	13
4.6 Labels and Source Indices	13

Chapter 5 — What Is Analysed	14
5.1 Dataset Inventory	14
5.2 Character and Byte Counts	14
5.3 Token and Word Counts	14
5.4 Sentence, Paragraph, and Line Counts	14
5.5 Word-Length Statistics	14
5.6 Sentence-Length and Paragraph Statistics	14
5.7 Lexical Diversity and Type-Token Ratio	14
5.8 Hapax Legomena	14
5.9 Stopword Statistics	15
5.10 Top Tokens, Bigrams, and Trigrams	15
5.11 Punctuation, Digits, Uppercase, and Whitespace	15
5.12 Non-ASCII, Emoji, URLs, Email, Hashtags, and Mentions	15
5.13 Repeated-Line Fraction	15
5.14 Empty, Very-Short, and Very-Long Flags	15
5.15 Readability Estimates	15
5.16 Writing-Script Detection	16
5.17 Exact Duplicate Detection	16
5.18 TF-IDF Pairwise Distance	16
Chapter 6 — Visualisations	17
6.1 plot_dataset() — Comprehensive Dataset Dashboard	17
Row 1: Dataset Overview and Label Distribution	17
Row 2: Basic Length and Lexical Diversity	17
Row 3: Sentence/Word Length, Stopwords, and Readability	17
Row 4: Symbol and Repetition Rates	17
Row 5: Top Words and Bigrams	17
Row 6: Scripts, Formats, and Scatter Diagnostics	17
Row 7: Pairwise Distances and Quality Flags	17
6.2 plot() — Single-Document Deep Dive	19
Document Overview and Text Preview	19
Top Tokens and Top Bigrams	19
Token and Sentence Length	19
Character Composition and Token Coverage	20
Script, Quality, and Sentence Sequence	20
6.3 plot_word_frequency() — Dataset Word Frequency	20
6.4 plot_ngram_frequency() — N-gram Frequency	20
6.5 plot_length_distribution() — Length Dashboard	21
6.6 plot_label_comparison() — Metric by Class	22
6.7 plot_pairwise_document_distances() — Similarity Heatmap	22

6.8 plot_text_samples() — Text Sample Grid	23
Chapter 7 — The TextRecord Object.....	25
7.1 Identity and Status Fields.....	25
7.2 Count Fields	25
7.3 Length and Distribution Fields	25
7.4 Lexical Fields	25
7.5 Character and Symbol Fields.....	26
7.6 Quality and Style Fields	26
Chapter 8 — HTML Report, CLI, and Testing	27
8.1 HTML Report Structure	27
8.2 CLI Options.....	27
8.3 User-Dataset Test Workflow.....	27
8.4 Common Errors.....	27
Chapter 9 — Advanced Usage.....	29
9.1 Custom Stopwords.....	29
9.2 Preserve Case.....	29
9.3 Minimum Token Length.....	29
9.4 Thresholds for Short and Long Documents	29
9.5 Access Records as a pandas DataFrame.....	29
9.6 Dataset Split Hygiene.....	30
9.7 Compare Two Corpora.....	30
9.8 Large Corpora.....	30
Chapter 10 — Interpreting Results and Common Findings	31
10.1 Red Flags to Watch For	31
10.2 Interpreting the Attached Example Dataset	31
10.3 Readability Is Not Model Quality	31
10.4 Token Counts Are Not Transformer Tokens	31
Chapter 11 — API Reference	32
11.1 Return and Failure Behaviour.....	33
Chapter 12 —Roadmap.....	34
Version 1.0.0 — TextEDA Integration	34
Roadmap	34
Appendix A — Complete TextRecord Field Reference	35
Appendix B — summary() Schema.....	37
Appendix C — Mathematical and Heuristic Reference	38
Appendix D — Writing Scripts Recognised.....	39
Appendix E — Glossary	40
Appendix F — Acknowledgements	41

Chapter 1 — Introduction

1.1 What is VisEDA TextEDA?

TextEDA is the text and natural-language exploratory analysis module of VisEDA. One TextEDA object can analyse a single document, a directory of text files, a list of file paths, structured CSV/TSV/JSON/JSONL records, or a sequence of in-memory strings. Every document is converted into a TextRecord containing counts, lexical statistics, symbol rates, readability estimates, writing-script information, quality flags, and a short preview. Dataset-level methods aggregate these records and analyse corpus vocabulary and document similarity.

1.2 Why Text EDA Matters

- **Class balance:** sentiment, intent, topic, toxicity, or other labels can be heavily imbalanced even when the corpus looks large.
- **Length distribution:** short messages and long articles have very different truncation, padding, batching, and memory requirements.
- **Vocabulary coverage:** a corpus with many singleton tokens, identifiers, spelling variants, or domain terms may require different tokenisation and normalisation choices.
- **Duplicates and near-duplicates:** repeated documents can leak across train/validation/test splits and inflate evaluation scores.
- **Encoding and script variation:** UTF-8, UTF-16, legacy encodings, and multilingual scripts can create silent parsing failures or unintended language mixtures.
- **Markup and symbols:** URLs, email addresses, hashtags, mentions, emoji, punctuation, and numbers may be informative features or preprocessing noise depending on the task.
- **Stopword dominance:** very common function words can obscure content-word patterns in frequency analyses.
- **Readability and sentence structure:** large variation in syntactic length or reading difficulty can reveal domain shift between classes or data sources.
- **Repeated boilerplate:** templates, disclaimers, headers, and repeated lines can dominate document similarity without representing the target task.
- **Corrupt or empty records:** empty fields and parsing errors can propagate to tokenisers and model training unless detected early.

1.3 Design Philosophy

- **Lightweight by default.** Tokenisation, sentence splitting, script analysis, readability, duplicate hashing, and most plots work without large NLP frameworks.
- **Structured and unstructured inputs.** The same class accepts individual files, class-folder datasets, CSV/TSV tables, JSON/JSONL corpora, and Python strings.
- **Progressive disclosure.** `summary()` provides statistics; `plot_dataset()` and `plot()` provide visual interpretation; `report()` provides a shareable HTML document.
- **Transparent heuristics.** Readability, sentence splitting, stopwords, and near-duplicate thresholds are explicitly documented rather than hidden behind a model.
- **Optional scikit-learn acceleration.** TF-IDF cosine distances use scikit-learn when available and fall back to a NumPy implementation when it is not.
- **User-controlled vocabulary behaviour.** Case folding, minimum token length, and the stopwords set are constructor options.
- **Dataset integrity first.** Empty, very-short, very-long, duplicate, encoding, and script anomalies are treated as EDA findings rather than silently discarded.

1.4 Installation

```
# Core VisEDA/TextEDA requirements
pip install viseda numpy matplotlib
```

```
# Recommended for TF-IDF cosine distances
pip install scikit-learn

# Required only to run this Word documentation generator
pip install python-docx pillow
```

Note: TextEDA does not require NLTK, spaCy, or transformer libraries for its built-in analysis. If scikit-learn is unavailable, `pairwise_document_distances()` uses the included NumPy TF-IDF fallback.

1.5 Supported Data Formats

Table 1. Supported TextEDA input formats.

Extension	Interpretation	Document expansion behaviour
.txt / .text	Plain text	One file becomes one document.
.md / .rst / .log	Plain textual content	One file becomes one document; markup is not stripped.
.html / .htm	Web text	HTML tags are removed; script/style/noscript/SVG content is ignored.
.csv	Comma-delimited structured data	Each data row becomes a document using a selected text field.
.tsv	Tab-delimited structured data	Each data row becomes a document using a selected text field.
.json	JSON string, object, list, or wrapper object	Objects/lists can expand into multiple documents.
.jsonl / .ndjson	One JSON value per line	Each non-empty line is parsed and expanded.
Python strings	In-memory documents	Use <code>load_texts()</code> with optional labels and names.

1.6 Tokenisation and Language Scope

The default tokenizer is Unicode-aware and is implemented with a regular expression. It recognises alphabetic words, including internal apostrophes and hyphens, and numeric tokens that may contain commas or decimal points. When `lowercase=True`, tokens are case-folded. `min_token_length` removes short tokens except numeric tokens.

The built-in stopword list and Flesch readability heuristic are English-oriented. Writing-script analysis is multilingual, but TextEDA does not perform language identification, stemming, lemmatisation, POS tagging, named-entity recognition, sentiment classification, or transformer tokenisation.

Chapter 2 — Quick Start

2.1 Directory Dataset with Parent-Folder Labels

```
# Example structure
# text_dataset/
#   positive/
#     review_001.txt
#     review_002.txt
#   negative/
#     review_003.txt
#   neutral/
#     review_004.txt

from viseda import TextEDA

eda = TextEDA(verbose=True)
eda.load(
    "text_dataset/",
    label_from_parent=True,
    recursive=True,
)

stats = eda.summary()
print(stats["inventory"])
print(stats["lexical"])
```

2.2 CSV, TSV, JSON, and JSONL Datasets

```
from viseda import TextEDA

eda = TextEDA()
eda.load(
    "reviews.csv",
    text_field="text",
    label_field="label",
)

# The same arguments apply to .tsv, .json, .jsonl, and .ndjson
print(eda.summary()["labels"])
```

Note: When `text_field` or `label_field` is omitted, `TextEDA` attempts to identify common field names automatically. If a text field cannot be determined, it raises a `KeyError` listing the available fields.

2.3 In-Memory Text

```
from viseda import TextEDA

texts = [
    "The bridge inspection report describes surface cracking.",
    "Traffic congestion increased during the evening period.",
]
labels = ["infrastructure", "traffic"]

eda = TextEDA()
eda.load_texts(texts, labels=labels)
```

2.4 Getting a Summary

```
summary = eda.summary()

print(summary["inventory"])
print(summary["length"])
print(summary["lexical"])
print(summary["symbols"])
```

```
print(summary["readability"])
print(summary["quality"])
print(summary["labels"])
```

2.5 Vocabulary and N-gram Analysis

```
# Ordered token -> frequency dictionary
top_words = eda.vocabulary(top_n=30)

# Exclude the configured stopwords list
content_words = eda.vocabulary(
    top_n=30,
    exclude_stopwords=True,
)

print(top_words)
```

2.6 Document Similarity and Near-Duplicates

```
distances, names = eda.pairwise_document_distances(
    max_documents=50,
    max_features=5000,
    exclude_stopwords=True,
)

similar_pairs = eda.near_duplicate_pairs(
    threshold=0.15,
    max_documents=100,
)
```

2.7 Generate Visualisations

```
eda.plot_dataset()           # comprehensive corpus dashboard
eda.plot(document_index=0)   # one-document deep dive
eda.plot_word_frequency()    # unigram frequency
eda.plot_ngram_frequency(n=2) # bigram frequency
eda.plot_length_distribution() # length-focused dashboard
eda.plot_label_comparison("word_count") # per-label box plot
eda.plot_pairwise_document_distances() # TF-IDF distance heatmap
eda.plot_text_samples(n=12, cols=3) # text preview grid

# Save instead of display
eda.plot_dataset(
    save_path="outputs_text/plot_dataset.png",
    dpi=160,
)
```

2.8 Generate an HTML Report

```
report_path = eda.report(
    "outputs_text/viseda_text_report.html"
)
print(report_path)
```

2.9 Command-Line Interface

```
# Directory dataset
viseda text "C:/data/text_dataset" --label-from-parent --plot --save-plot outputs_text/plot_dataset.png --
report outputs_text/report.html

# Structured CSV
viseda text "C:/data/reviews.csv" --text-field text --label-field label --plot
```

2.10 User-Dataset-Only Test Script

```
# Directory
python testtext.py --dir "C:/Users/Ike/Desktop/test_dataset/folder_dataset" --save-plots

# CSV
python testtext.py --file "C:/data/reviews.csv" --text-field text --label-field label --save-plots
```

Note: The supplied testtext.py is intentionally user-dataset-only. It does not generate or analyse a synthetic corpus. The HTML report and plots therefore correspond to the dataset passed with --dir or --file.

Chapter 3 — The TextEDA Class

3.1 Constructor

```
def __init__(self, verbose=True, max_documents=None, lowercase=True, min_token_length=1, stopwords=None,
short_document_words=5, long_document_words=1000, top_n=30, encoding=None, random_seed=0) -> None:
```

Table 2. TextEDA constructor parameters.

Parameter	Default	Purpose
verbose	True	Print loading and reporting progress.
max_documents	None	Cap the number of expanded documents analysed.
lowercase	True	Case-fold tokens for vocabulary/frequency analyses.
min_token_length	1	Minimum token length; numeric tokens are retained even when short.
stopwords	None	Optional iterable replacing the built-in English stopwords set.
short_document_words	5	Documents with fewer tokens are flagged very_short.
long_document_words	1000	Documents with more tokens are flagged very_long.
top_n	30	Number of top unigrams, bigrams, and trigrams retained per TextRecord.
encoding	None	Preferred input encoding tried before automatic fallbacks.
random_seed	0	Seed used by reproducible sampling in internal fallback workflows.

3.2 load() — Files, Lists, and Directories

```
def load( self, source: Union[str, Path, Sequence[Union[str, Path]]], labels: Optional[Mapping[str, str]] =
None, label_from_parent: bool = False, recursive: bool = True, text_field: Optional[str] = None, label_field:
Optional[str] = None, ) -> "TextEDA":
```

load() resolves a file, a list of files, or a directory. Only files whose extensions appear in SUPPORTED_EXTS are accepted. Directories are searched recursively by default. A path-to-label mapping can be supplied through labels; otherwise label_from_parent=True uses each file's immediate parent directory.

Structured files can expand one physical file into many TextRecord objects. In those cases, source_index identifies the row or JSON item and the record name is written as filename#source_index. max_documents applies after expansion.

3.3 load_texts() — In-Memory Strings

```
def load_texts( self, texts: Sequence[str], labels: Optional[Sequence[str]] = None, names:
Optional[Sequence[str]] = None, ) -> "TextEDA":
```

load_texts() is useful for data already held in pandas, databases, APIs, notebooks, or preprocessing pipelines. Each string becomes a TextRecord with file_ext='text' and encoding='unicode'.

3.4 get_record() and get_text()

```
rec = eda.get_record(0)
text = eda.get_text(0)

print(rec.word_count)
print(rec.lexical_diversity)
print(text)
```

get_record() exposes the computed TextRecord. get_text() retrieves the original string stored by the TextEDA object. Both methods require a completed load() or load_texts() call.

3.5 vocabulary() — Corpus Token Frequencies

```
def vocabulary( self, min_frequency: int = 1, top_n: Optional[int] = None, exclude_stopwords: bool = False, ) -> Dict[str, int]:
```

vocabulary() aggregates the same tokens used by TextEDA's record-level analysis. Items are sorted first by descending frequency and then alphabetically for equal counts. min_frequency removes rare tokens, top_n truncates the output, and exclude_stopwords uses the configured stopword set.

3.6 summary() — Full Dataset Statistics

```
def summary(self) -> Dict[str, Any]:
```

summary() returns seven top-level sections: inventory, length, lexical, symbols, readability, quality, and labels. Numeric document-level metrics are aggregated with count, mean, standard deviation, minimum, 25th percentile, median, 75th percentile, and maximum.

```
summary = eda.summary()

summary["inventory"]
summary["length"]
summary["lexical"]
summary["symbols"]
summary["readability"]
summary["quality"]
summary["labels"]
```

3.7 pairwise_document_distances()

```
def pairwise_document_distances( self, max_documents: int = 50, max_features: int = 5_000, exclude_stopwords: bool = True, ) -> Tuple[np.ndarray, List[str]]:
```

The preferred implementation uses scikit-learn TfidfVectorizer followed by cosine_distances. If scikit-learn is unavailable or the vectorisation step raises an exception, TextEDA builds TF-IDF vectors with NumPy and computes 1 - cosine similarity.

$$distance(i,j) = 1 - cosine(TFIDF_i, TFIDF_j)$$

Note: The matrix describes lexical similarity, not semantic embedding similarity. Two paraphrases with different vocabulary may remain far apart, while templated documents can appear very close.

3.8 near_duplicate_pairs()

```
def near_duplicate_pairs( self, threshold: float = 0.15, max_documents: int = 100, ) -> List[Tuple[str, str, float]]:
```

Every upper-triangular document pair whose TF-IDF cosine distance is less than or equal to threshold is returned. The default 0.15 threshold corresponds to a cosine similarity of at least 0.85.

3.9 report() — Self-Contained HTML

```
def report(self, output_path: str = "visedatextreport.html") -> str:
```

report() writes a single HTML file with embedded CSS and no external assets. It includes corpus inventory, label/format/script distributions, length statistics, symbol and markup statistics, readability, quality, vocabulary statistics, top words, and top bigrams.

Chapter 4 — Input Parsing and Dataset Organisation

4.1 Path Resolution

For a directory, TextEDA uses `rglob('*')` when `recursive=True` and `glob('*')` otherwise, then keeps only files with supported extensions. A single unsupported file raises `ValueError`. A missing path raises `FileNotFoundError`.

4.2 Encoding Detection

TextEDA attempts the user-specified encoding first when provided, then tries UTF-8, UTF-8 with BOM, UTF-16, UTF-16 little-endian, UTF-16 big-endian, Windows CP1252, and Latin-1. The first successful decoding is stored in `TextRecord.encoding`.

```
# Force an encoding preference when known
eda = TextEDA(encoding="utf-8")
eda.load("corpus/")
```

4.3 Plain Text and Markup Files

.txt, .text, .md, .rst, and .log are read as raw text. Markdown and reStructuredText syntax is therefore included in punctuation and token statistics. HTML/HTM is treated differently: tags are removed and script, style, noscript, and SVG content is ignored by the built-in HTML parser.

4.4 CSV and TSV Field Selection

Delimited files are read with `csv.DictReader`. If `text_field` is given, it must match an existing column exactly. Otherwise TextEDA searches the following candidates case-insensitively.

Table 3. Automatic structured-data field detection.

Purpose	Automatically recognised field names
Text	text, content, document, sentence, review, comment, body, description, message, abstract, article, question
Label	label, class, category, target, sentiment, topic, intent

Note: If a structured dataset contains only one column and no `text_field` is specified, that sole column is used as text. Label detection is optional; if no candidate label field exists, documents remain unlabelled.

4.5 JSON and JSONL Expansion

A JSON string is treated as one document. A JSON list is iterated. A JSON object is first checked for list-valued wrapper keys data, records, items, documents, or examples; otherwise the object itself is treated as one item. JSONL/NDJSON parses each non-empty line independently and records its line number as `source_index`.

4.6 Labels and Source Indices

Structured labels extracted from `label_field` take precedence over a base label supplied by `labels` or `label_from_parent`. For plain files, the base label is retained. This makes folder labels and row-level labels composable without changing the public loading interface.

Chapter 5 — What Is Analysed

5.1 Dataset Inventory

Inventory statistics include total, valid, and corrupt document counts; corrupt paths; file-format distribution; detected encoding distribution; label distribution; and dominant writing-script distribution.

5.2 Character and Byte Counts

`char_count` is Python's Unicode string length. `byte_count` is the length after UTF-8 encoding with replacement for encoding errors. For ASCII-only text these values are typically equal; multilingual or emoji-rich text can require multiple UTF-8 bytes per character.

5.3 Token and Word Counts

`word_count` stores the total number of regex tokens, including numeric tokens. `alpha_word_count` counts tokens that fully match the alphabetic `WORD_RE` pattern. `numeric_token_count` counts tokens containing at least one digit. `unique_word_count` counts distinct analysed tokens after case folding and minimum-length filtering.

5.4 Sentence, Paragraph, and Line Counts

Sentence splitting is heuristic: sentence boundaries occur after '.', '!', or '?' followed by whitespace, or at newline boundaries. Paragraphs are separated by one or more blank lines. `line_count` uses the original line structure, while `nonempty_line_count` ignores whitespace-only lines.

Note: Sentence segmentation is intentionally lightweight. Abbreviations, quotations, ellipses, URLs, and language-specific punctuation can produce different boundaries from a trained sentence segmenter.

5.5 Word-Length Statistics

For every record TextEDA stores average, median, and standard deviation of token lengths. `Dataset.summary()` aggregates `avg_word_length` across documents. These metrics can reveal identifiers, codes, URLs fragmented into tokens, or domain-specific long terminology.

5.6 Sentence-Length and Paragraph Statistics

Sentence lengths are measured in analysed tokens. The record stores average, median, standard deviation, and maximum sentence length. `avg_paragraph_words` is the mean token count of detected paragraphs.

5.7 Lexical Diversity and Type-Token Ratio

$$\text{lexical_diversity} = \text{unique_tokens} / \text{total_tokens}$$

The same ratio is calculated per document and at corpus level. Values close to 1 indicate that most tokens occur only once; lower values indicate heavier reuse. The measure is strongly length-dependent, so comparisons should account for document size.

5.8 Hapax Legomena

$$\text{hapax_ratio} = \text{tokens_with_frequency_1} / \text{unique_tokens}$$

A hapax token appears exactly once. High hapax ratios can indicate rich vocabulary, many identifiers, spelling noise, proper names, OCR artefacts, or a very small corpus. TextEDA reports document-level hapax counts/ratios and corpus-level hapax statistics.

5.9 Stopword Statistics

$$\text{stopword_ratio} = \text{stopword_tokens} / \text{total_tokens}$$

The default stopwords set is a compact built-in English list. It can be completely replaced through the stopwords constructor parameter. Stopword ratios are diagnostic rather than prescriptive: some NLP tasks benefit from retaining stopwords.

5.10 Top Tokens, Bigrams, and Trigrams

Each TextRecord retains its top_n unigrams, bigrams, and trigrams. Dataset summary() additionally returns the top 50 words, content words, bigrams, and trigrams. N-grams are contiguous token sequences without smoothing or linguistic filtering.

5.11 Punctuation, Digits, Uppercase, and Whitespace

$$\text{punctuation_rate} = \text{punctuation_characters} / \text{characters}$$

$$\text{digit_rate} = \text{digit_characters} / \text{characters}$$

$$\text{uppercase_rate} = \text{uppercase_letters} / \text{alphabetic_characters}$$

$$\text{whitespace_rate} = \text{whitespace_characters} / \text{characters}$$

Punctuation is defined as a character that is neither alphanumeric nor whitespace. These rates can reveal code-like data, templated messages, unusual capitalisation, or numeric-heavy records.

5.12 Non-ASCII, Emoji, URLs, Email, Hashtags, and Mentions

non_ascii_rate measures the fraction of characters whose code point exceeds 127. Regex counters separately identify emoji, HTTP/WWW URLs, email addresses, hashtags, and @mentions. These are simple counts and do not redact or transform the underlying text.

5.13 Repeated-Line Fraction

Non-empty lines are whitespace-normalised and case-folded before comparison. repeated_line_fraction is one minus the fraction of unique normalised lines. It is designed to identify repeated boilerplate, logs, templated content, or accidental line duplication.

$$\text{repeated_line_fraction} = \frac{1 - \text{unique_normalised_lines}}{\text{nonempty_lines}}$$

5.14 Empty, Very-Short, and Very-Long Flags

empty is true when text.strip() is empty. very_short is true when word_count is below short_document_words. very_long is true when word_count exceeds long_document_words. The defaults are 5 and 1000 tokens respectively and are configurable.

5.15 Readability Estimates

TextEDA uses a lightweight English syllable heuristic and computes two familiar readability estimates. Only alphabetic tokens are used in the syllable calculation. Documents without alphabetic words or detected sentences receive None for readability.

$$\text{Flesch} = 206.835 - 1.015(W/S) - 84.6(Y/W)$$

W = alphabetic words, S = sentences, Y = estimated syllables.

$$\text{FK_grade} = 0.39(W/S) + 11.8(Y/W) - 15.59$$

Note: These formulas and the included syllable heuristic are English-oriented. For multilingual corpora, use readability as an exploratory flag rather than a universal linguistic score.

5.16 Writing-Script Detection

Alphabetic characters are assigned to Unicode ranges. The implementation recognises Latin, Cyrillic, Greek, Arabic, Hebrew, Devanagari, Bengali, Thai, CJK, Hiragana, Katakana, Hangul, and Other. `script_distribution` gives per-document fractions and `dominant_script` records the largest group.

5.17 Exact Duplicate Detection

Before hashing, all whitespace is normalised to single spaces, and the text is case-folded. TextEDA then calculates a SHA-256 hash. Documents sharing the same hash form an exact-duplicate group even if their original spacing or letter case differs.

Note: Because normalisation ignores whitespace and case, 'Exact duplicate' here means exact after TextEDA's normalisation, not byte-for-byte identical files.

5.18 TF-IDF Pairwise Distance

The scikit-learn path uses `TfidfVectorizer` with `min_df=1`, a Unicode word token pattern, the configured lowercase setting, optional stopword removal, and `max_features`. The NumPy fallback computes term frequency, smoothed inverse document frequency, L2 normalisation, cosine similarity, and finally `distance = 1 - similarity`.

$$idf(t) = \log((1 + N) / (1 + df(t))) + 1$$

Chapter 6 — Visualisations

6.1 `plot_dataset()` — Comprehensive Dataset Dashboard

```
def plot_dataset( self, figsize: Tuple[int, int] = (22, 28), save_path: Optional[str] = None, dpi: int = 160, top_n: int = 20, ) -> None:
```

Row 1: Dataset Overview and Label Distribution

The overview card reports total and valid documents, corrupt count, unique labels, mean word count, vocabulary size, type-token ratio, and exact-duplicate fraction. The adjacent horizontal bar chart visualises class balance.

Row 2: Basic Length and Lexical Diversity

Histograms show word count, character count, sentence count, and document lexical diversity. Dashed vertical lines indicate the dataset mean when data are available.

Row 3: Sentence/Word Length, Stopwords, and Readability

The third row shows average sentence length, average word length, stopword ratio, and Flesch Reading Ease. These panels make it easier to identify stylistic or preprocessing differences between documents.

Row 4: Symbol and Repetition Rates

Punctuation rate, digit rate, non-ASCII rate, and repeated-line fraction highlight code-like, numeric, multilingual, or templated records.

Row 5: Top Words and Bigrams

Corpus-level unigrams and bigrams are presented as horizontal frequency bars. By default stopwords are retained, which allows the plot to reveal how dominant function words are; use dedicated frequency methods with `exclude_stopwords=True` for content-word analysis.

Row 6: Scripts, Formats, and Scatter Diagnostics

The writing-script and file-format distributions are accompanied by Words vs Unique Words and Average Sentence Length vs Readability scatter plots. These expose document-level relationships that histograms may hide.

Row 7: Pairwise Distances and Quality Flags

The final row displays a TF-IDF cosine-distance matrix for up to 30 documents and fractions of empty, very-short, very-long, and exact-duplicate documents.



6.2 plot() — Single-Document Deep Dive

```
def plot( self, document_index: int = 0, figsize: Tuple[int, int] = (18, 15), save_path: Optional[str] = None,
        dpi: int = 160, top_n: int = 20, ) -> None:
```



Figure 6-2. TextEDA Single-Document Dashboard

A deep dive into one document: overview card, text preview, top tokens, token and sentence-length distributions, bigrams, character-category rates, cumulative token coverage, script composition, quality/composition flags, and sentence-length sequence.

Document Overview and Text Preview

The overview card includes path/name, label, word count, unique words, sentence and paragraph counts, lexical diversity, stopword ratio, readability, and dominant script. The text preview shows the original document within a bounded panel.

Top Tokens and Top Bigrams

Frequency bars show the dominant token vocabulary and contiguous two-token phrases in the selected document.

Token and Sentence Length

Token-length and sentence-length histograms reveal unusually long tokens, terse statements, or long sentences within the selected record.

Character Composition and Token Coverage

Character rates summarise punctuation, digits, uppercase, whitespace, and non-ASCII content. The cumulative token-coverage curve shows how quickly the most frequent vocabulary items account for the document's token mass.

Script, Quality, and Sentence Sequence

The final row displays the document's writing-script proportions, short/long/repeated-line/stopword/non-ASCII composition, and a sentence-by-sentence word-count sequence.

6.3 plot_word_frequency() — Dataset Word Frequency

```
def plot_word_frequency( self, top_n: int = 30, exclude_stopwords: bool = False, figsize: Tuple[int, int] = (12, 8), save_path: Optional[str] = None, dpi: int = 160, ) -> None:
```

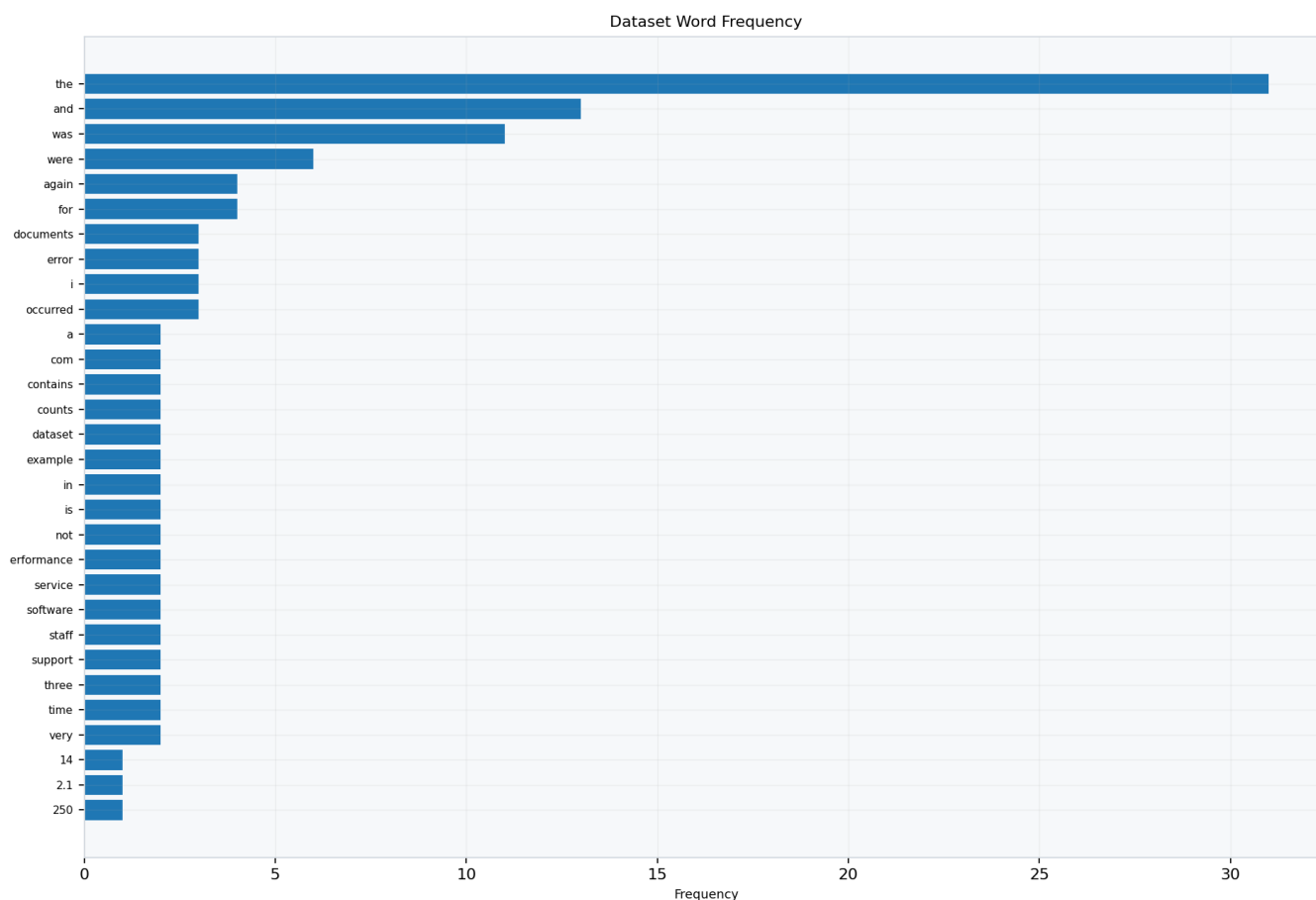


Figure 6-3. Dataset Word Frequency

A horizontal bar chart of the most frequent corpus tokens. The supplied example retains stopwords, so common function words appear prominently.

For topic-oriented interpretation, rerun with `exclude_stopwords=True`. For preprocessing audits, keeping stopwords can be more informative because it shows the raw lexical composition.

6.4 plot_ngram_frequency() — N-gram Frequency

```
def plot_ngram_frequency( self, n: int = 2, top_n: int = 30, exclude_stopwords: bool = False, figsize: Tuple[int, int] = (12, 8), save_path: Optional[str] = None, dpi: int = 160, ) -> None:
```

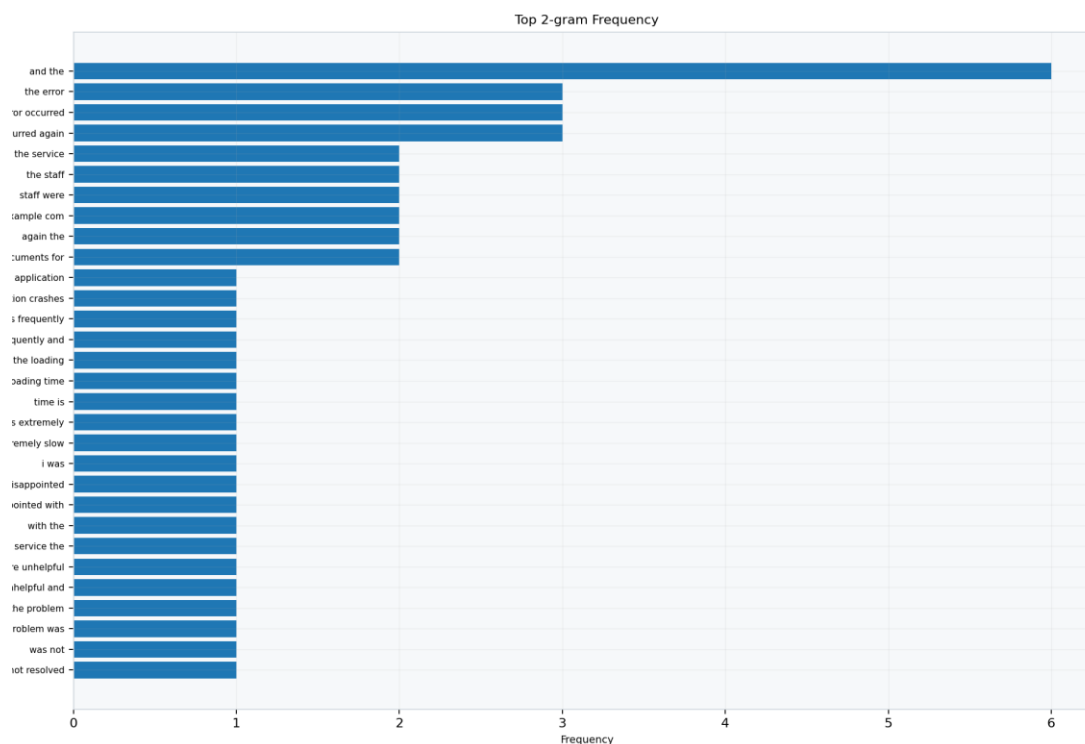


Figure 6-4. Top N-gram Frequency

Frequency of the most common contiguous n-token sequences. The attached example uses $n=2$ and therefore shows bigrams.

6.5 plot_length_distribution() — Length Dashboard

```
def plot_length_distribution( self, figsize: Tuple[int, int] = (16, 10), save_path: Optional[str] = None, dpi:
    int = 160, ) -> None:
```

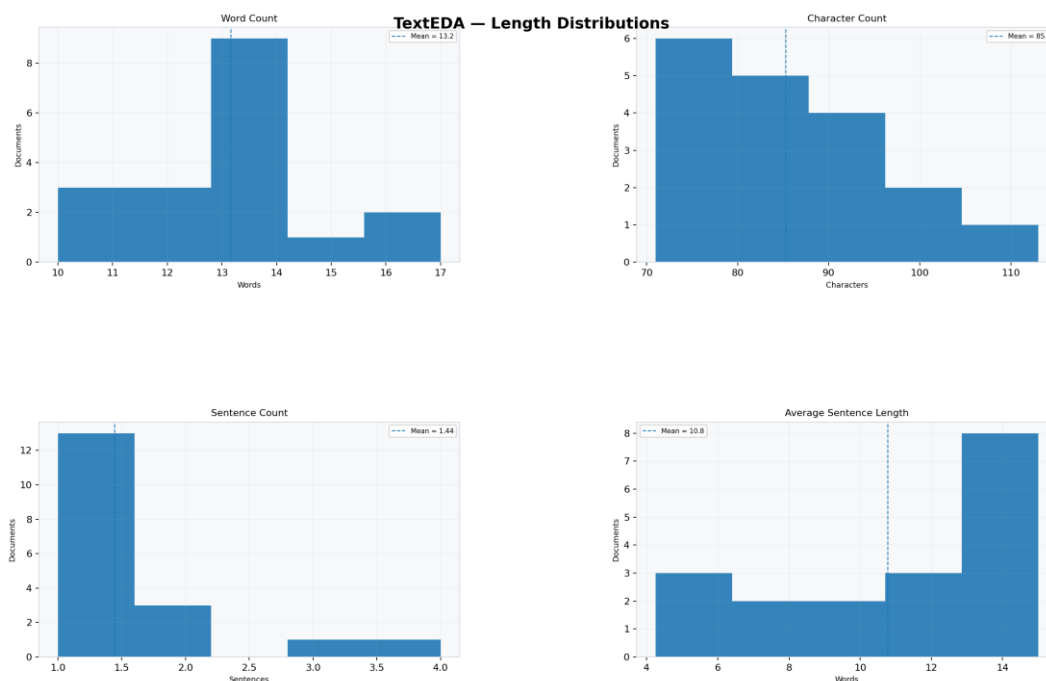


Figure 6-5. Text Length Distributions

Four histograms summarising word count, character count, sentence count, and average sentence length across documents.

6.6 plot_label_comparison() — Metric by Class

```
def plot_label_comparison( self, metric: str = "word_count", figsize: Tuple[int, int] = (12, 7), save_path:
Optional[str] = None, dpi: int = 160, ) -> None:
```

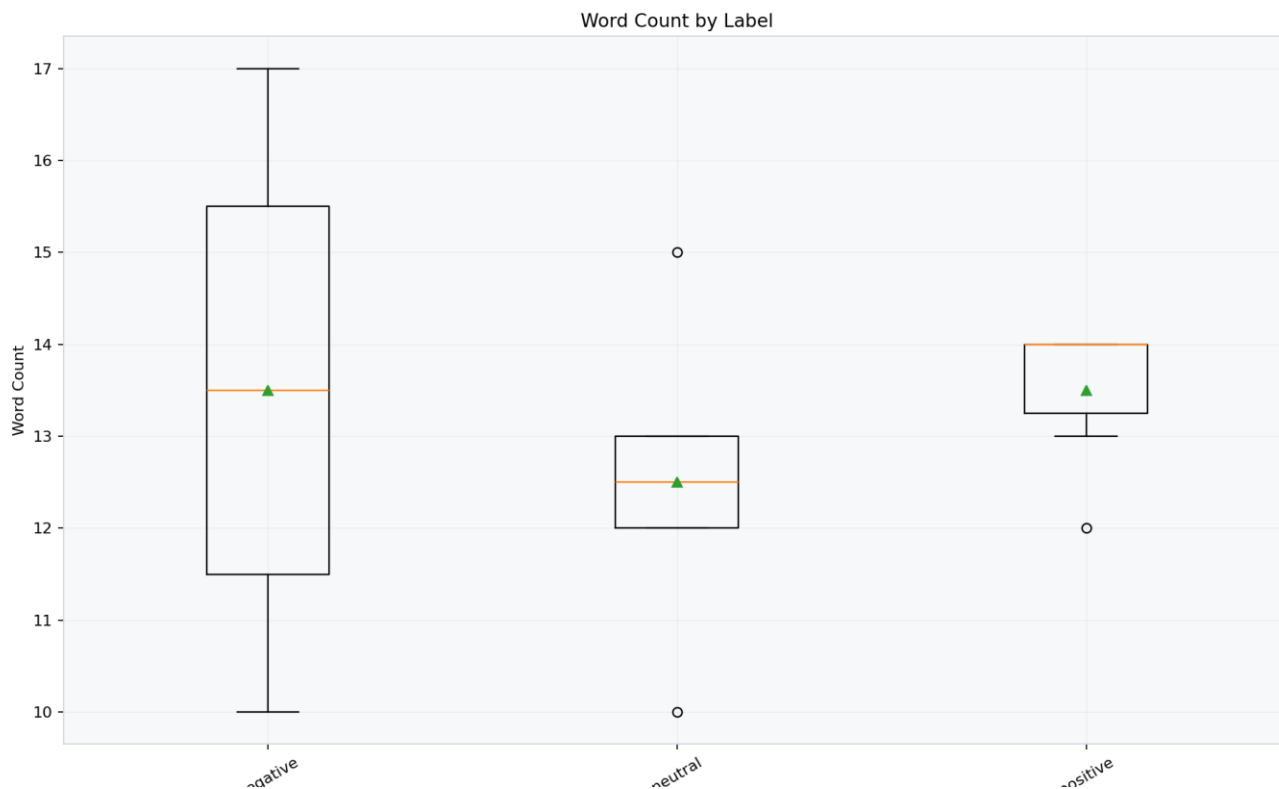


Figure 6-6. Word Count by Label

A box plot comparing a numeric TextRecord metric across labels. The mean is shown in addition to standard box-plot statistics.

metric can be any existing numeric TextRecord attribute, for example word_count, lexical_diversity, avg_sentence_length, stopwords_ratio, punctuation_rate, or readability_flesch. Unknown attributes raise ValueError.

6.7 plot_pairwise_document_distances() — Similarity Heatmap

```
def plot_pairwise_document_distances( self, max_documents: int = 50, figsize: Tuple[int, int] = (11, 9),
save_path: Optional[str] = None, dpi: int = 160, ) -> None:
```

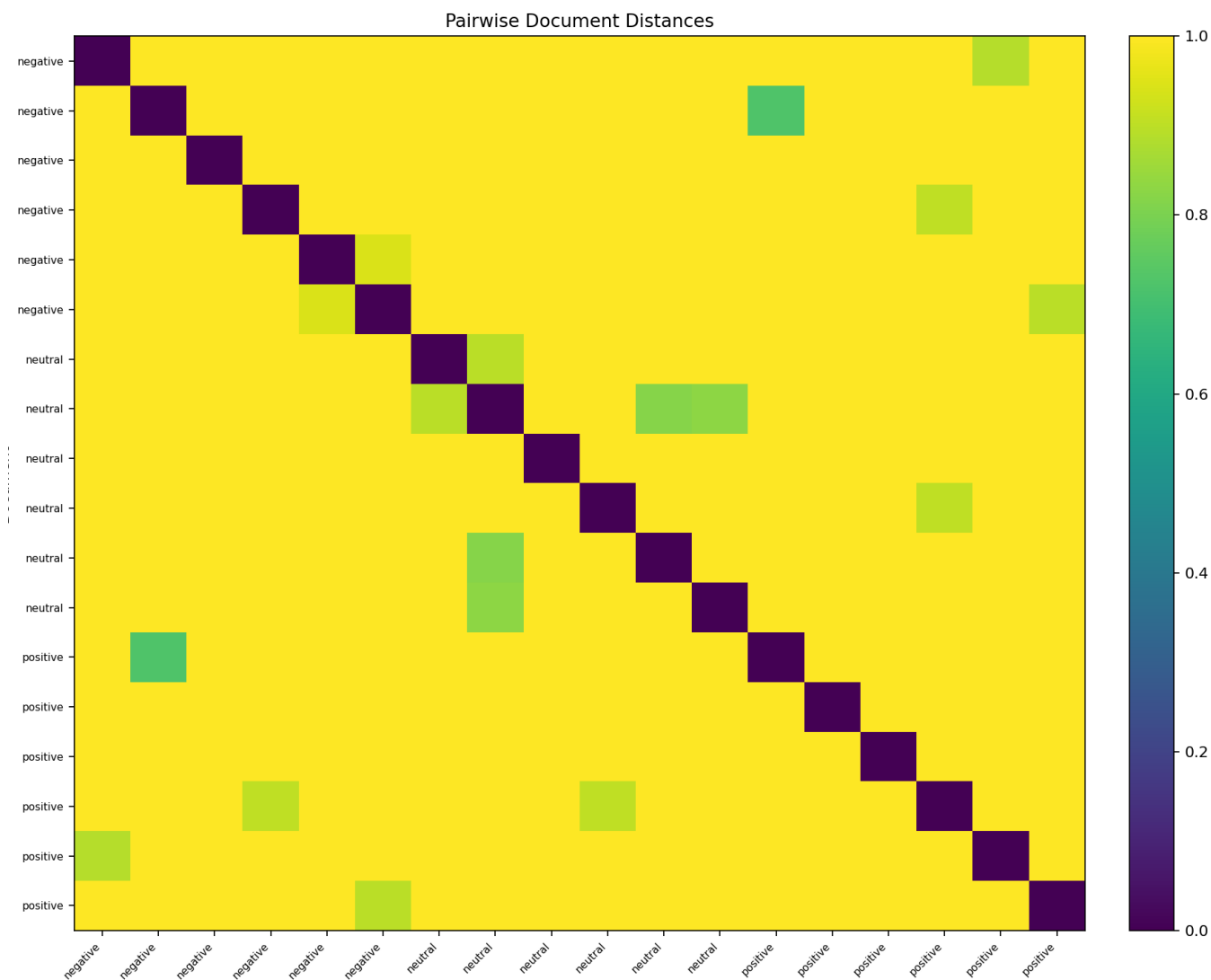


Figure 6-7. Pairwise Document Distances

A heatmap of TF-IDF cosine distances. Zero-valued diagonal cells represent each document compared with itself; smaller off-diagonal values indicate stronger lexical similarity.

Note: If multiple records share the same label, axis labels can repeat. The matrix order follows the loaded record order; use `get_record()` or custom names with `load_texts()` when unique display names are needed.

6.8 `plot_text_samples()` — Text Sample Grid

```
def plot_text_samples( self, n: int = 12, cols: int = 3, chars: int = 450, figsize: Optional[Tuple[int, int]] =
None, save_path: Optional[str] = None, dpi: int = 160, ) -> None:
```

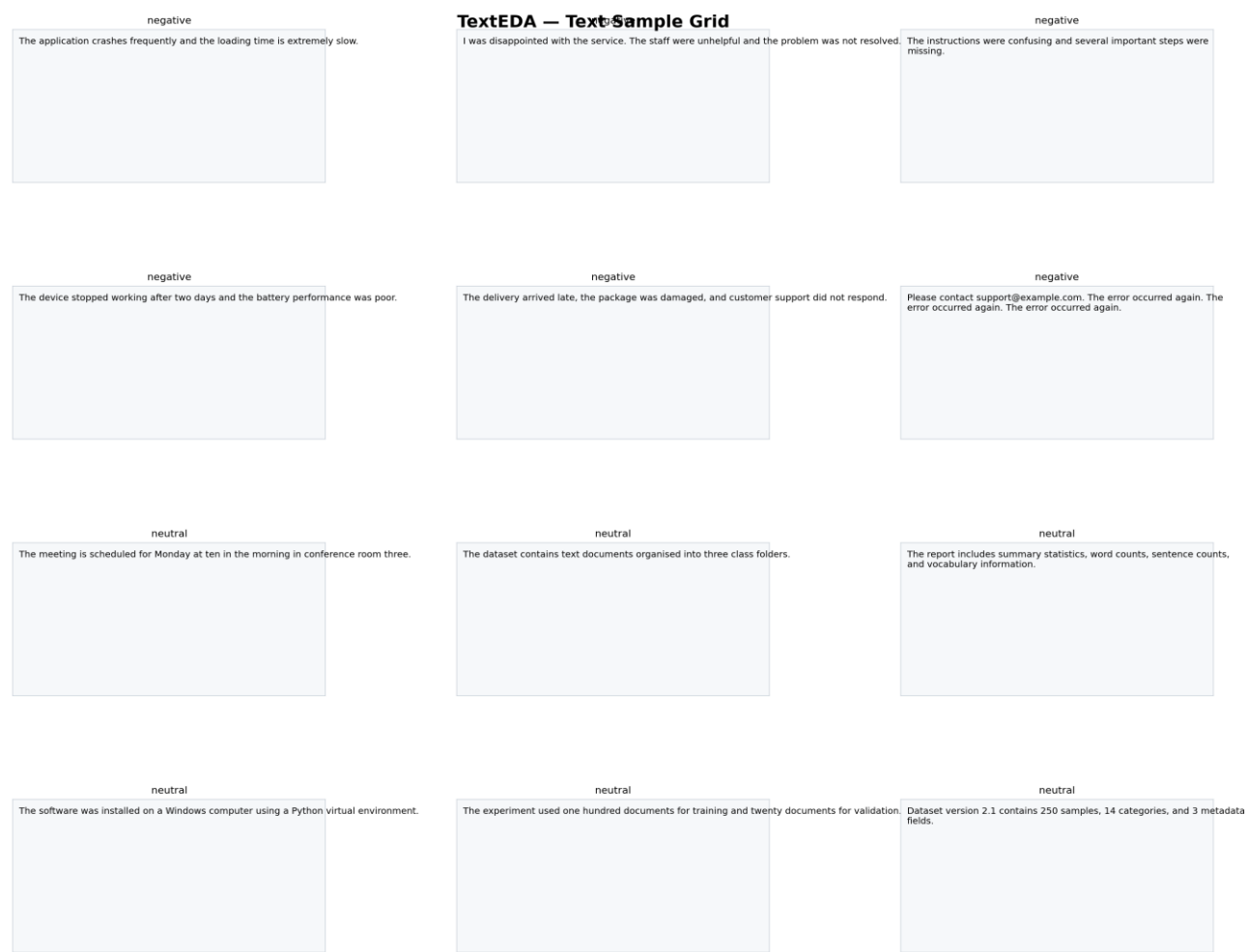


Figure 6-8. TextEDA Text Sample Grid

A compact visual audit of up to n valid documents. Each panel displays a normalised preview and the document label or name.

The grid is useful for detecting incorrect labels, repeated templates, unexpected languages, formatting artefacts, and content that numerical statistics alone may not reveal.

Chapter 7 — The TextRecord Object

Every loaded or expanded document produces one TextRecord. The class uses `__slots__` to define the complete field schema. Fields are grouped below by function.

7.1 Identity and Status Fields

Field	Meaning
path	Source path or in-memory name.
name	Display name; structured rows use filename#source_index.
label	Structured label, mapped label, parent-folder label, or None.
file_ext	Input extension or 'text' for load_texts().
source_index	Row/item/line index for expanded structured inputs.
file_size_kb	Physical file size in kilobytes when file-backed.
encoding	Successful decoding used for the source file.
is_corrupt	True when reading or record analysis failed.
error	Stored error message for corrupt records.
normalised_hash	SHA-256 hash after whitespace normalisation and case folding.

7.2 Count Fields

Field	Meaning
char_count	Unicode character count.
byte_count	UTF-8 encoded byte count.
word_count	Total analysed tokens, including numeric tokens.
alpha_word_count	Alphabetic word tokens.
numeric_token_count	Tokens containing at least one digit.
unique_word_count	Distinct analysed tokens.
sentence_count	Heuristically detected sentences.
paragraph_count	Blank-line separated paragraphs.
line_count	Number of source lines.
nonempty_line_count	Source lines containing non-whitespace content.

7.3 Length and Distribution Fields

Field	Meaning
avg_word_length	Mean analysed token length.
median_word_length	Median token length.
word_length_std	Token-length standard deviation.
avg_sentence_length	Mean tokens per detected sentence.
median_sentence_length	Median tokens per sentence.
sentence_length_std	Sentence-length standard deviation.
avg_paragraph_words	Mean tokens per detected paragraph.
max_sentence_words	Largest detected sentence token count.

7.4 Lexical Fields

Field	Meaning
lexical_diversity	unique_word_count / word_count.
hapax_count	Number of token types occurring once in the document.
hapax_ratio	hapax_count / unique_word_count.

Field	Meaning
stopword_count	Configured stopwords occurrences.
stopword_ratio	stopword_count / word_count.
top_tokens	Most common unigrams retained up to top_n.
top_bigrams	Most common contiguous 2-grams.
top_trigrams	Most common contiguous 3-grams.

7.5 Character and Symbol Fields

Field	Meaning
punctuation_count / rate	Non-alphanumeric, non-whitespace characters and their character fraction.
digit_count / rate	Digit characters and their character fraction.
uppercase_count / rate	Uppercase letters and fraction of alphabetic characters.
whitespace_count / rate	Whitespace characters and their character fraction.
non_ascii_count / rate	Characters with code point > 127 and their fraction.
emoji_count	Matches from the built-in emoji Unicode ranges.
url_count	HTTP/HTTPS/WWW URL matches.
email_count	Email-address regex matches.
hashtag_count	#token matches.
mention_count	@token matches.

7.6 Quality and Style Fields

Field	Meaning
repeated_line_fraction	Fraction of non-empty lines duplicated after normalisation.
empty	Whether stripped text is empty.
very_short	Whether word_count is below short_document_words.
very_long	Whether word_count exceeds long_document_words.
readability_flesch	Flesch Reading Ease estimate or None.
readability_grade	Flesch-Kincaid grade estimate or None.
script_distribution	Fractions of alphabetic characters by recognised Unicode script.
dominant_script	Script with the largest alphabetic-character count.
preview	Whitespace-normalised first 500 characters.

Chapter 8 — HTML Report, CLI, and Testing

8.1 HTML Report Structure

The generated HTML uses a responsive light-theme card layout. Inventory cards show total/valid/corrupt documents together with label, format, and dominant-script distributions. Subsequent sections summarise length, symbols and markup, readability, quality, and vocabulary.

The attached report was generated from an 18-document example corpus and correctly reports 18 total/valid documents with six negative, six neutral, and six positive labels. It demonstrates that `report()` reflects the currently loaded user dataset rather than a built-in fixture.

8.2 CLI Options

CLI option	Meaning
<code>path</code>	Text file or dataset directory.
<code>--max</code>	Maximum documents to analyse.
<code>--text-field</code>	Text column/key for CSV/TSV/JSON.
<code>--label-field</code>	Label column/key for CSV/TSV/JSON.
<code>--label-from-parent</code>	Use the parent folder as label.
<code>--encoding</code>	Preferred input encoding.
<code>--preserve-case</code>	Disable token case folding.
<code>--min-token-length</code>	Minimum token length.
<code>--short-words</code>	Very-short document threshold.
<code>--long-words</code>	Very-long document threshold.
<code>--report</code>	Write an HTML report.
<code>--plot</code>	Generate a dataset or single-document dashboard.
<code>--single-plot</code>	Plot one document rather than the dataset.
<code>--document-index</code>	Document index for <code>--single-plot</code> .
<code>--save-plot</code>	Save the selected dashboard image.

8.3 User-Dataset Test Workflow

The supplied `testtext.py` requires either `--dir` or `--file`. It validates loading, summary output, required `TextRecord` fields, vocabulary, document distances, all applicable visualisations, and HTML report generation. No synthetic dataset is generated or analysed.

```
python testtext.py --dir "C:/Users/Ike/Desktop/test_dataset/folder_dataset" --save-plots

# Optional output controls
python testtext.py --file "C:/data/reviews.jsonl" --text-field text --label-field label --save-plots
--output-dir outputs_text --report viseda_text_report.html
```

8.4 Common Errors

Error / symptom	Likely cause	Resolution
Unsupported text format	File extension not in <code>SUPPORTED_EXTS</code> .	Convert the file or extend TextEDA's loader.
Could not identify a text field	Structured file has no recognised text column/key.	Pass <code>text_field/--text-field</code> explicitly.
Requested <code>label_field</code> not found	Specified label column/key is absent.	Check field spelling or omit the label field.
Unable to decode	None of the encoding fallbacks succeeded.	Pass encoding explicitly or convert the file to Unicode.
Need at least two valid	Distance analysis was requested on	Load at least two documents or skip

Error / symptom	Likely cause	Resolution
documents	one valid document.	pairwise analysis.
No labelled documents are available	plot_label_comparison() has no labels.	Provide labels via folders, mappings, or structured label_field.
Unknown TextRecord metric	Invalid metric name in plot_label_comparison().	Use a numeric TextRecord attribute.
Readability is None	No alphabetic words/sentences were available.	Treat as unavailable rather than zero.
DOCX PermissionError	Generated Word file is open/locked.	Close it; the documentation generator also supports timestamp fallback.

Chapter 9 — Advanced Usage

9.1 Custom Stopwords

```
domain_stopwords = {
    "the", "and", "of",
    "report", "document", "section",
}

eda = TextEDA(stopwords=domain_stopwords)
eda.load("corpus/")

content_vocab = eda.vocabulary(
    top_n=50,
    exclude_stopwords=True,
)
```

Note: Passing stopwords replaces the built-in set; it does not extend it automatically. Combine sets explicitly when you want both general and domain-specific stopwords.

9.2 Preserve Case

```
eda = TextEDA(lowercase=False)
eda.load("named_entities/")

# 'Apple' and 'apple' remain separate vocabulary entries.
print(eda.vocabulary(top_n=30))
```

9.3 Minimum Token Length

```
eda = TextEDA(min_token_length=3)
eda.load("corpus/")
```

Short alphabetic tokens below the threshold are removed from vocabulary and length analyses. Numeric tokens remain eligible even when their string length is shorter than the threshold.

9.4 Thresholds for Short and Long Documents

```
eda = TextEDA(
    short_document_words=20,
    long_document_words=512,
)
eda.load("documents/")
```

Choose these thresholds to match the modelling pipeline. For example, a transformer with a 512-token context does not map exactly to 512 TextEDA regex tokens, but TextEDA can still flag obviously long records for closer inspection.

9.5 Access Records as a pandas DataFrame

```
import pandas as pd

rows = []
for rec in eda._records:
    if rec.is_corrupt:
        continue
    rows.append({
        "path": rec.path,
        "label": rec.label,
        "words": rec.word_count,
        "sentences": rec.sentence_count,
        "lexical_diversity": rec.lexical_diversity,
        "stopword_ratio": rec.stopword_ratio,
```

```

        "readability": rec.readability_flesch,
        "script": rec.dominant_script,
    })

df = pd.DataFrame(rows)
df.to_csv("texteda_records.csv", index=False)

```

9.6 Dataset Split Hygiene

Before random train/validation/test splitting, inspect `exact_duplicate_groups` and `near_duplicate_pairs()`. Documents from the same template, source article, conversation, or duplicated record should generally remain in one split when leakage would invalidate evaluation.

9.7 Compare Two Corpora

```

train = TextEDA()
train.load("train/", label_from_parent=True)

test = TextEDA()
test.load("test/", label_from_parent=True)

a = train.summary()
b = test.summary()

print("Train mean words:", a["length"]["words"]["mean"])
print("Test mean words:", b["length"]["words"]["mean"])
print("Train vocab:", a["lexical"]["dataset_unique_tokens"])
print("Test vocab:", b["lexical"]["dataset_unique_tokens"])
print("Train scripts:", a["inventory"]["dominant_script_distribution"])
print("Test scripts:", b["inventory"]["dominant_script_distribution"])

```

9.8 Large Corpora

`max_documents` limits the expanded corpus. Pairwise distances independently cap analysis with `max_documents` because the distance matrix grows quadratically in document count. For very large corpora, use `summary()` on a representative sample and run similarity analysis on stratified subsets.

pairwise matrix storage grows as $O(N^2)$

Chapter 10 — Interpreting Results and Common Findings

10.1 Red Flags to Watch For

Finding	Possible implication	Next action
High corrupt count	Unsupported encoding or malformed structured data.	Inspect <code>corrupt_paths</code> and source-file errors.
Many empty documents	Missing fields or failed upstream extraction.	Remove/fix empty records before tokenisation.
Many very-short documents	Labels may be inferred from insufficient context.	Review thresholds and task requirements.
Many very-long documents	Truncation or memory pressure likely.	Plan chunking, summarisation, or long-context models.
High exact-duplicate fraction	Train/test leakage and corpus redundancy risk.	Deduplicate or group before splitting.
Small pairwise distances	Near-duplicate or templated records.	Inspect text pairs and source provenance.
Very high type-token ratio	Tiny corpus, many rare terms, IDs, spelling/OCR noise.	Inspect hapax and top content words.
High repeated-line fraction	Boilerplate, logs, headers, or accidental duplication.	Strip templates or repeated metadata if irrelevant.
Unexpected dominant scripts	Mixed language or encoding content.	Stratify by language/script or audit source files.
Large class differences in length	Length may become a shortcut feature.	Use <code>plot_label_comparison</code> and inspect sampling/preprocessing.
Readability differs by label	Classes may be confounded with writing style/source.	Audit source/domain distribution.
High URL/email/mention rates	Social/web metadata may dominate vocabulary.	Decide whether to normalise, mask, or preserve these tokens.

10.2 Interpreting the Attached Example Dataset

The attached example report contains 18 valid .txt documents divided evenly among negative, neutral, and positive labels. The balanced label bars therefore do not indicate class imbalance, while the length and lexical plots demonstrate how TextEDA can still reveal within-class variation even when counts are balanced.

The pairwise distance heatmap is predominantly high-distance away from the diagonal, with a small number of lower-distance pairs. This suggests most documents have distinct lexical content while some share wording. The quality-flags panel shows no empty, very-short, very-long, or exact-duplicate records under the configured thresholds.

Note: Findings from the demonstration corpus illustrate how to read the plots; they are not universal quality targets. Thresholds and distributions must be interpreted relative to the NLP task and data source.

10.3 Readability Is Not Model Quality

Flesch scores describe surface-level English reading difficulty, not semantic correctness, sentiment, toxicity, factuality, or usefulness for a language model. Treat readability as a dataset-characterisation metric.

10.4 Token Counts Are Not Transformer Tokens

TextEDA's regex tokens do not correspond to BPE, WordPiece, SentencePiece, or other subword tokenisers. Use TextEDA for broad corpus EDA, then measure the exact production tokenizer when context-window utilisation is critical.

Chapter 11 — API Reference

Method	Current signature	Purpose
<code>__init__</code>	<code>TextEDA(verbose=True, max_documents=None, lowercase=True, min_token_length=1, stopwords=None, short_document_words=5, long_document_words=1000, top_n=30, encoding=None, random_seed=0)</code>	Create a text-analysis session.
<code>load</code>	<code>def load(self, source: Union[str, Path, Sequence[Union[str, Path]]], labels: Optional[Mapping[str, str]] = None, label_from_parent: bool = False, recursive: bool = True, text_field: Optional[str] = None, label_field: Optional[str] = None,) -> "TextEDA":</code>	Load supported files/directories.
<code>load_texts</code>	<code>def load_texts(self, texts: Sequence[str], labels: Optional[Sequence[str]] = None, names: Optional[Sequence[str]] = None,) -> "TextEDA":</code>	Load in-memory strings.
<code>get_record</code>	<code>def get_record(self, index: int = 0) -> TextRecord:</code>	Return one TextRecord.
<code>get_text</code>	<code>def get_text(self, index: int = 0) -> str:</code>	Return stored source text.
<code>vocabulary</code>	<code>def vocabulary(self, min_frequency: int = 1, top_n: Optional[int] = None, exclude_stopwords: bool = False,) -> Dict[str, int]:</code>	Return sorted token frequencies.
<code>summary</code>	<code>def summary(self) -> Dict[str, Any]:</code>	Return nested corpus statistics.
<code>pairwise_document_distances</code>	<code>def pairwise_document_distances(self, max_documents: int = 50, max_features: int = 5_000, exclude_stopwords: bool = True,) -> Tuple[np.ndarray, List[str]]:</code>	Return TF-IDF cosine distances and names.
<code>near_duplicate_pairs</code>	<code>def near_duplicate_pairs(self, threshold: float = 0.15, max_documents: int = 100,) -> List[Tuple[str, str, float]]:</code>	Return close document pairs.
<code>plot_dataset</code>	<code>def plot_dataset(self, figsize: Tuple[int, int] = (22, 28), save_path: Optional[str] = None, dpi: int = 160, top_n: int = 20,) -> None:</code>	Comprehensive dataset dashboard.
<code>plot</code>	<code>def plot(self, document_index: int = 0, figsize: Tuple[int, int] = (18, 15), save_path: Optional[str] = None, dpi: int = 160, top_n: int = 20,) -> None:</code>	Single-document dashboard.
<code>plot_word_frequency</code>	<code>def plot_word_frequency(self, top_n: int = 30, exclude_stopwords: bool = False, figsize: Tuple[int, int] = (12, 8), save_path: Optional[str] = None, dpi: int = 160,) -> None:</code>	Corpus unigram plot.
<code>plot_ngram_frequency</code>	<code>def plot_ngram_frequency(self, n: int = 2, top_n: int = 30, exclude_stopwords: bool = False, figsize: Tuple[int, int] = (12, 8), save_path: Optional[str] = None, dpi: int = 160,) -> None:</code>	Corpus n-gram plot.
<code>plot_length_distribution</code>	<code>def plot_length_distribution(self, figsize: Tuple[int, int] = (16, 10), save_path: Optional[str] = None, dpi: int = 160,) -> None:</code>	Length dashboard.
<code>plot_label_comparison</code>	<code>def plot_label_comparison(self, metric: str = "word_count", figsize: Tuple[int, int] = (12, 7), save_path: Optional[str] = None, dpi: int = 160,) -> None:</code>	Compare a metric by label.
<code>plot_pairwise_document_distances</code>	<code>def plot_pairwise_document_distances(self, max_documents: int = 50, figsize: Tuple[int, int] = (11, 9), save_path: Optional[str] = None, dpi: int = 160,) -> None:</code>	Distance heatmap.
<code>plot_text_samples</code>	<code>def plot_text_samples(self, n: int = 12, cols: int = 3, chars: int = 450, figsize: Optional[Tuple[int, int]] = None,</code>	Document preview grid.

Method	Current signature	Purpose
	save_path: Optional[str] = None, dpi: int = 160,) -> None:	
report	def report(self, output_path: str = "visedata_text_report.html") -> str:	Write HTML report and return path.

11.1 Return and Failure Behaviour

load() and load_texts() return self. get_record() returns TextRecord; get_text() returns str; vocabulary() and summary() return dictionaries; pairwise_document_distances() returns a NumPy matrix plus display names; near_duplicate_pairs() returns a list of tuples; plotting methods return None; report() returns the saved path string.

Analysis methods require a loaded object and raise RuntimeError if called before load()/load_texts(). Pairwise distance requires at least two valid documents. Invalid field names, unsupported formats, and missing paths raise explicit exceptions.

Chapter 12 —Roadmap

Version 1.0.0 — TextEDA Integration

- Unified TextEDA class for files, directories, structured corpora, and in-memory strings.
- TextRecord schema covering length, lexical, symbol, readability, script, and quality metrics.
- Plain text, HTML, CSV, TSV, JSON, JSONL, and NDJSON loading.
- Automatic structured text/label field detection.
- Encoding fallback chain and corrupt-record tracking.
- Corpus vocabulary, n-grams, exact duplicates, TF-IDF distances, and near-duplicate pairs.
- Comprehensive dataset dashboard and single-document dashboard.
- Dedicated frequency, length, label-comparison, distance, and sample-grid plots.
- Self-contained HTML report.
- User-dataset-only test workflow without synthetic-corpus reporting.

Roadmap

- Language identification and language-conditioned summaries.
- Pluggable spaCy/NLTK sentence and word tokenisers while retaining the lightweight default.
- Transformer-token length profiling for selected Hugging Face tokenisers.
- Embedding-based semantic similarity and duplicate detection.
- Named-entity, POS, dependency, and phrase-distribution analysis as optional extensions.
- Unicode normalisation diagnostics (NFC/NFD/NFKC/NFKD).
- PII-pattern auditing with privacy-preserving reporting.
- Topic-modelling and clustering visualisations.
- Class-conditional vocabulary and distinctive-term analysis.
- Train/validation/test corpus comparison utilities.
- Streaming/iterator interfaces for corpora too large to retain in memory.
- Document source/provenance metadata support.
- Interactive HTML dashboards and searchable text-sample tables.

Appendix A — Complete TextRecord Field Reference

Field	Group	Meaning
path	Identity	Source path or in-memory identifier.
name	Identity	Display name for the record.
label	Identity	Optional class/target label.
file_ext	Identity	Source extension or text.
source_index	Identity	Expanded structured-data index.
file_size_kb	Identity	Source-file size in KB.
encoding	Identity	Successful text decoding.
is_corrupt	Status	Whether reading/analysis failed.
error	Status	Stored failure text.
normalised_hash	Quality	Whitespace/case-normalised SHA-256.
char_count	Counts	Unicode characters.
byte_count	Counts	UTF-8 bytes.
word_count	Counts	Analysed tokens.
alpha_word_count	Counts	Alphabetic word tokens.
numeric_token_count	Counts	Tokens containing digits.
unique_word_count	Counts	Distinct analysed tokens.
sentence_count	Counts	Detected sentences.
paragraph_count	Counts	Detected paragraphs.
line_count	Counts	All source lines.
nonempty_line_count	Counts	Non-empty source lines.
avg_word_length	Length	Mean token characters.
median_word_length	Length	Median token characters.
word_length_std	Length	Token-length standard deviation.
avg_sentence_length	Length	Mean tokens/sentence.
median_sentence_length	Length	Median tokens/sentence.
sentence_length_std	Length	Sentence-length standard deviation.
avg_paragraph_words	Length	Mean tokens/paragraph.
max_sentence_words	Length	Maximum tokens in a sentence.
lexical_diversity	Lexical	Unique/total token ratio.
hapax_count	Lexical	Token types occurring once.
hapax_ratio	Lexical	Hapax/unique ratio.
stopword_count	Lexical	Configured stopword occurrences.
stopword_ratio	Lexical	Stopwords/total tokens.
top_tokens	Lexical	Top unigrams.
top_bigrams	Lexical	Top contiguous bigrams.
top_trigrams	Lexical	Top contiguous trigrams.
punctuation_count/rate	Symbols	Punctuation count and character fraction.
digit_count/rate	Symbols	Digit count and character fraction.
uppercase_count/rate	Symbols	Uppercase count and alphabetic fraction.
whitespace_count/rate	Symbols	Whitespace count and character fraction.
non_ascii_count/rate	Symbols	Non-ASCII count and character fraction.
emoji_count	Symbols	Emoji regex matches.
url_count	Symbols	URL regex matches.
email_count	Symbols	Email regex matches.
hashtag_count	Symbols	Hashtag matches.
mention_count	Symbols	Mention matches.
repeated_line_fraction	Quality	Repeated normalised non-empty line fraction.
empty	Quality	Empty-document flag.
very_short	Quality	Very-short threshold flag.

Field	Group	Meaning
very_long	Quality	Very-long threshold flag.
readability_flesch	Readability	Flesch Reading Ease estimate.
readability_grade	Readability	Flesch-Kincaid grade estimate.
script_distribution	Script	Alphabetic-character fractions by script.
dominant_script	Script	Most frequent recognised script.
preview	Preview	Normalised first 500 characters.

Appendix B — summary() Schema

```
{
  "inventory": {
    "total_documents", "valid_documents",
    "corrupt_documents", "corrupt_paths",
    "format_distribution", "encoding_distribution",
    "label_distribution", "dominant_script_distribution"
  },
  "length": {
    "characters", "bytes", "words", "sentences",
    "paragraphs", "lines", "avg_word_length",
    "avg_sentence_length", "sentence_length_std"
  },
  "lexical": {
    "dataset_total_tokens", "dataset_unique_tokens",
    "dataset_type_token_ratio", "dataset_hapax_count",
    "dataset_hapax_ratio", "document_lexical_diversity",
    "document_hapax_ratio", "stopword_ratio",
    "top_words", "top_content_words",
    "top_bigrams", "top_trigrams"
  },
  "symbols": {
    "punctuation_rate", "digit_rate", "uppercase_rate",
    "whitespace_rate", "non_ascii_rate", "emoji_count",
    "url_count", "email_count", "hashtag_count",
    "mention_count"
  },
  "readability": {
    "flesch_reading_ease", "flesch_kincaid_grade"
  },
  "quality": {
    "empty_documents", "very_short_documents",
    "very_long_documents", "repeated_line_fraction",
    "exact_duplicate_groups", "documents_in_duplicate_groups",
    "exact_duplicate_fraction"
  },
  "labels": {
    "label_distribution", "class_imbalance_ratio"
  }
}
```

Appendix C — Mathematical and Heuristic Reference

Metric / operation	Definition used by TextEDA
Lexical diversity	unique tokens / total tokens
Hapax ratio	token types occurring once / unique tokens
Stopword ratio	stopword tokens / total tokens
Punctuation rate	punctuation characters / characters
Digit rate	digit characters / characters
Uppercase rate	uppercase letters / alphabetic characters
Whitespace rate	whitespace characters / characters
Non-ASCII rate	characters with <code>ord(ch)>127</code> / characters
Repeated-line fraction	$1 - \text{unique normalised non-empty lines} / \text{non-empty lines}$
Flesch Reading Ease	$206.835 - 1.015(W/S) - 84.6(Y/W)$
Flesch-Kincaid grade	$0.39(W/S) + 11.8(Y/W) - 15.59$
Fallback TF	token frequency / total document tokens
Fallback IDF	$\log((1+N)/(1+df)) + 1$
Pairwise distance	$1 - \text{cosine similarity of TF-IDF vectors}$
Exact duplicate hash	SHA-256 of whitespace-normalised, case-folded text

Appendix D — Writing Scripts Recognised

Script	Unicode coverage in the current implementation
Latin	Basic/Extended Latin ranges 0041-024F and 1E00-1EFF
Cyrillic	0400-052F
Greek	0370-03FF
Arabic	0600-06FF, 0750-077F, 08A0-08FF
Hebrew	0590-05FF
Devanagari	0900-097F
Bengali	0980-09FF
Thai	0E00-0E7F
CJK	3400-4DBF, 4E00-9FFF, F900-FAFF
Hiragana	3040-309F
Katakana	30A0-30FF
Hangul	AC00-D7AF and 1100-11FF
Other	Alphabetic characters outside the listed ranges

Appendix E — Glossary

Term	Definition
Corpus	A collection of text documents analysed as one dataset.
Token	A unit returned by TextEDA's regular-expression tokenizer.
Type	A unique token value in a vocabulary.
Type-token ratio	Number of unique token types divided by total tokens.
Hapax legomenon	A token type occurring exactly once.
Stopword	A common function word included in the configured exclusion set.
N-gram	A contiguous sequence of n tokens.
TF-IDF	Term-frequency inverse-document-frequency weighting.
Cosine distance	One minus cosine similarity between vector representations.
Near-duplicate	Documents whose TF-IDF distance is below a selected threshold.
Exact duplicate	Documents sharing TextEDA's normalised SHA-256 hash.
Readability	Surface-level estimate of reading difficulty; not a semantic quality measure.
Writing script	Unicode character system such as Latin, Arabic, CJK, or Hangul.
Encoding	Mapping used to decode bytes into Unicode text.
Boilerplate	Repeated template text such as headers, disclaimers, or signatures.
Structured text dataset	Rows/items where text and labels are stored in named fields.

Appendix F — Acknowledgements

TextEDA is implemented with Python's standard text-processing libraries, NumPy, Matplotlib, and optional scikit-learn. The module is intended to support researchers and practitioners working in NLP, text classification, sentiment analysis, information retrieval, document analysis, social-media analytics, and language-model dataset preparation.